

STONK GEN — Launch App

A working front-end that connects Phantom wallet and mints a real SPL token on Solana (devnet by default, mainnet-beta optional).

What's actually real here

- **Wallet connect** — uses Phantom's injected provider (`window.solana`).
- **Token minting** — creates a real mint account and mints your full supply to your wallet. Standard SPL Token Program launch.
- **Locked Launch (mandatory, not a toggle)** — mint and freeze authority are always revoked in the same transaction as the mint itself, and the launch terms (name, ticker, supply, decimals, parent token, launching wallet, timestamp) are written on-chain via the SPL Memo program *before* the mint instruction, in the same atomic transaction. This directly answers the "vamping" problem that hit Pump.fun in 2026 — creators editing fee structures post-launch to drain liquidity. Here there's nothing left to edit: the terms are on-chain and the memo is public and permanent.
- **Generations / lineage** — a launch can name another token's mint address as its parent. That relationship is written into the same on-chain memo, so lineage is independently verifiable by anyone reading the transaction — not just a label your app shows.
- **Receipt** — shows the mint address, transaction signature, lock status, parent token (if any), and a link to Solana Explorer.

The six tabs

- **Explore** — every coin launched from *this browser* (stored in `localStorage` , not a global feed — see below), shown with a computed Generation badge based on parent links.
- **Launch** — the mint flow described above.
- **Creators** — connect a wallet to see just that wallet's launch history and a simple track record (coins launched, % locked — always 100% since locking isn't optional here).

- **Token Lab** — paste any STONK GEN transaction signature and it fetches the real transaction from the chain, decodes the on-chain memo itself (hand-rolled base58 decode, no trust in what this app displays elsewhere), and shows the locked terms plus a count of Token Program instructions. This is genuinely independent verification, not a lookup against local storage.
- **Narrative** — a small rule-based name/ticker generator. It's explicitly not AI — a static page can't safely hold an API key, so this is a placeholder for a real backend-backed generator later.
- **Docs** — static explanation of Locked Launch and Generations, and what still needs a backend.

What's NOT included yet (and needs more than a static page)

- **Lineage revenue-sharing** — right now, naming a parent token just records the relationship on-chain. Actually routing a slice of a child coin's trading fees back to its parent's community (the "family tree funds itself" idea) needs a real bonding-curve/AMM program with configurable fee splits — the same category of infrastructure Bags built on Meteora's Dynamic Bonding Curve. That's a proper Anchor/Rust program plus a liquidity design, not a front-end feature. Worth scoping as its own project once the base launch flow is proven out.
- **On-chain metadata** (name/symbol/image showing up in Phantom, Solscan, etc.) — the token exists and is fully functional, but wallets need a separate metadata step to display its name/logo. Two options:
 - **Metaplex Token Metadata** — the most widely supported; add `@metaplex-foundation/mpl-token-metadata` and a `createMetadataAccountV3` instruction to the transaction.
 - **Token-2022 metadata extension** — newer, on-mint metadata with no separate account, but fewer wallets fully support it yet.
- **Explore feed / live market caps / creator pages / generation tree** — Explore and Creators right now only show launches made from the *same browser* (via `localStorage`), so nothing is missed on this device but nothing from other people shows up either. A real cross-device feed, live market caps, and full generation trees need an indexer watching the chain (or a database you write to on launch) plus a backend API — either your own indexing service or a provider like Helius/Bitquery.
- **Narrative Generator, Token Lab, Pre-Signing Scanner (human-readable**

simulation) — these are product features on top of this base, not infrastructure. The scanner in particular is worth prioritizing: services like Blowfish or a Solana `simulateTransaction` call can show a plain-English preview before the user signs.

Running it

This is a single static file — no build step.

- **Locally:** just open `index.html` in a browser (Phantom must be installed as an extension).
- **Deployed:** drag the folder into [Vercel](#) or [Netlify](#), or push it to a GitHub repo and connect it — either will give you a live URL in a couple of minutes.

Before mainnet

- Test thoroughly on devnet first (free faucet SOL: `solana airdrop 2` via Solana CLI, or a web faucet).
- Consider a paid RPC endpoint (Helius, QuickNode, Triton) instead of the public `clusterApiUrl` — the public mainnet endpoint is rate-limited and not reliable for production traffic.
- Double check decimals and supply math — this app mints `supply × 10^decimals` base units, matching how SPL tokens actually work.